Guia Didático: Painel de Controle de Despesas

Projeto: aplicação em TypeScript para cadastrar, validar, listar, resumir e excluir despesas.

Este material explica a lógica do código por partes, com foco nos principais comandos de TypeScript e JavaScript utilizados no projeto. O objetivo é ajudar os alunos a entenderem não apenas *o que* o código faz, mas também *por que* cada função existe.

Fluxo geral da aplicação:

1. O usuário preenche o formulário.
2. O formulário é validado.
3. Uma despesa é criada e armazenada no array.
4. A lista visual é atualizada.
5. O resumo total e por categoria é recalculado.
6. O usuário pode excluir uma despesa.

Observação: no código enviado havia alguns trechos visualmente alterados por formatação/HTML (por exemplo, **randomUUID**). No PDF, a explicação considera a sintaxe correta do TypeScript, como **crypto.randomUUID()**.

1. IMPORTAÇÕES E TIPOS

```
import './style.css';
import type { Categoria, Despesa } from "./types";
```

import './style.css': importa o arquivo de estilos CSS para aplicar a aparência da página.

import type: importa apenas tipos do TypeScript. Esses tipos ajudam o editor e o compilador a verificar se os dados estão sendo usados corretamente.

Categoria: representa os valores permitidos para uma categoria.

Despesa: representa a estrutura de uma despesa, normalmente contendo id, título, valor e categoria.

2. CATEGORIAS E ARRAY DE DESPESAS

```
const CATEGORIAS: Categoria[] = [
  "Alimentos",
  "Transporte",
  "Lazer",
  "Saude",
  "Cartao",
  "Outros",
];

let despesas: Despesa[] = [];
```

const cria uma referência que não será substituída. O conteúdo de um array ainda pode ser utilizado normalmente.

Categoria[] significa: um array contendo valores do tipo **Categoria**.

let permite alterar a variável posteriormente. Isso é necessário porque o array **despesas** é substituído quando adicionamos ou removemos registros.

Inicialmente, **despesas = []**, ou seja, não existe nenhuma despesa cadastrada.

3. SELEÇÃO DOS ELEMENTOS HTML

```
const formularioNovaDespesa =
  document.querySelector<HTMLFormElement>("#formNovaDespesa")!;

const campoTituloDespesa =
  document.querySelector<HTMLInputElement>("#tituloDespesa")!;
```

document.querySelector() procura um elemento da página usando um seletor CSS. Por exemplo, **#tituloDespesa** procura um elemento com esse id.

O TypeScript permite informar o tipo esperado usando **<HTMLInputElement>**, **<HTMLFormElement>**, **<HTMLSelectElement>** etc.

O símbolo **!** no final informa ao TypeScript que o desenvolvedor tem certeza de que o elemento existe. Isso evita que o retorno seja tratado como **null**.

O mesmo conceito é utilizado para a mensagem de erro, lista de despesas, mensagem de lista vazia, soma total e totais por categoria.

4. GERAÇÃO DE ID

```
function gerarIdDespesa(): string {
  return crypto.randomUUID();
}
```

Cada despesa precisa de uma identificação única. **crypto.randomUUID()** gera um UUID, permitindo diferenciar uma despesa da outra.

O **: string** informa que a função sempre retorna um texto.

5. FORMATAÇÃO DE MOEDA

```
function formatarValorMoeda(valor: number): string {
  return valor.toLocaleString("pt-BR", {
    style: "currency",
    currency: "BRL"
  });
}
```

toLocaleString() formata um número de acordo com uma localização.

pt-BR aplica o padrão brasileiro. **style: 'currency'** informa que o resultado é moeda e **currency: 'BRL'** representa o Real brasileiro.

Exemplo: **1500.5** pode ser exibido como **R$ 1.500,50**.

6. SOMA DOS VALORES

```
function somarValoresDaLista(lista: Despesa[]): number {
  return lista.reduce(
    (soma, despesa) => soma + despesa.valor,
    0
  );
}
```

reduce() percorre o array e acumula um resultado.

A variável **soma** começa em **0**. Para cada despesa, o código adiciona **despesa.valor** ao acumulador.

Assim, se houver despesas de 10, 20 e 30, o resultado será 60.

7. TOTAIS POR CATEGORIA

```
const totais: Record<Categoria, number> = {
  Alimentos: 0,
  Transporte: 0,
  Lazer: 0,
  Saude: 0,
  Cartao: 0,
  Outros: 0,
};
```

Record<Categoria, number> representa um objeto em que cada categoria possui um valor numérico.

```
for (const despesa of lista) {
  totais[despesa.categoria] =
    totais[despesa.categoria] + despesa.valor;
}
```

for...of percorre diretamente cada item do array.

totais[despesa.categoria] acessa dinamicamente a categoria da despesa e adiciona o valor correspondente.

8. ADICIONAR UMA DESPESA

```
function adicionarDespesa(
  titulo: string,
  valor: number,
  categoria: Categoria
): Despesa | null {
  const tituloLimpo = titulo.trim();

  if (tituloLimpo.length === 0) {
    return null;
  }

  if (valor <= 0) {
    return null;
  }
}
```

trim() remove espaços no início e no final do texto.

if permite tomar decisões. Caso o título esteja vazio ou o valor seja menor ou igual a zero, a função retorna **null**.

Despesa | null significa que a função pode retornar uma despesa válida ou não retornar uma despesa.

```
const despesa: Despesa = {
  id: gerarIdDespesa(),
  titulo: tituloLimpo,
  valor: valor,
  categoria: categoria,
};

despesas = [...despesas, despesa];
```

O objeto **despesa** reúne todas as informações do novo registro.

O operador **...** é chamado de **spread operator**. Ele copia os itens existentes do array e adiciona a nova despesa ao final.

Essa abordagem evita modificar diretamente o array anterior.

9. VALIDAÇÃO DO FORMULÁRIO

```
const valorNumerico: number = Number(valor);
```

Os valores dos campos HTML são recebidos como texto. **Number(valor)** converte o conteúdo para número.

```
if (
  valor.trim().length === 0 ||
  Number.isNaN(valorNumerico) ||
  valorNumerico <= 0
) {
  return "Informe um valor válido para a despesa.";
}
```

|| significa **OU**. Basta uma das condições ser verdadeira para exibir o erro.

Number.isNaN() verifica se o resultado não é um número válido.

A função retorna uma mensagem de erro ou **null** quando tudo está correto.

10. EXIBIÇÃO DE ERROS

```
function exibirMensagemErro(mensagem: string | null): void {
  if (mensagem === null) {
    mensagemErroValidacao.hidden = true;
    return;
  }

  mensagemErroValidacao.textContent = mensagem;
  mensagemErroValidacao.hidden = false;
}
```

void indica que a função não retorna um valor.

textContent altera o texto de um elemento HTML.

hidden = true esconde o elemento; **hidden = false** volta a exibi-lo.

return encerra a função naquele ponto.

11. LIMPEZA DO FORMULÁRIO

```
function limparFormulario(): void {
  campoTituloDespesa.value = "";
  campoValorDespesa.value = "";
  campoCategoriaDespesa.value = "";
}
```

Após cadastrar uma despesa com sucesso, os campos são limpos para que o usuário possa realizar um novo cadastro.

12. CRIAÇÃO DO ITEM VISUAL DA DESPESA

```
const item: HTMLLIElement =
  document.createElement("li");

item.className = "despesa";
item.dataset.idDespesa = despesa.id;
```

document.createElement() cria elementos HTML usando JavaScript.

className define uma classe CSS.

dataset permite armazenar dados personalizados no elemento. Nesse caso, o id da despesa é armazenado para permitir sua exclusão posteriormente.

```
titulo.textContent = despesa.titulo;
categoria.textContent = despesa.categoria;

if (despesa.categoria === "Cartao") {
  categoria.textContent = "Cartão de Crédito";
}
```

O código utiliza **===**, comparação estrita, para verificar a categoria.

Internamente, a categoria pode ser **Cartao**, mas visualmente o usuário vê **Cartão de Crédito**.

```
botaoExcluir.setAttribute(
  "aria-label",
  `Excluir despesa: ${despesa.titulo}`
);
```

setAttribute() adiciona um atributo HTML.

aria-label melhora a acessibilidade, oferecendo uma descrição para tecnologias assistivas.

As crases ``` criam uma template string. O trecho **${...}** permite inserir valores dentro do texto.

13. ATUALIZAÇÃO DA LISTA

```
if (despesas.length === 0) {
  listaDespesas.textContent = "";
  mensagemListaVazia.hidden = false;
  return;
}
```

length informa a quantidade de itens do array.

Quando não existem despesas, a lista é limpa e a mensagem de lista vazia é exibida.

```
for (const despesa of despesas) {
  listaDespesas.append(
    criarItemDespesa(despesa)
  );
}
```

append() adiciona elementos ao HTML. Para cada despesa cadastrada, a função **criarItemDespesa()** gera um item visual.

14. ATUALIZAÇÃO DO RESUMO

```
const somaTotal = somarValoresDaLista(despesas);
elementoSomaTotal.textContent =
  formatarValorMoeda(somaTotal);
```

Primeiro, todas as despesas são somadas. Depois, o valor é formatado como moeda e exibido na página.

```
for (const categoria of CATEGORIAS) {
  // cria o item da categoria
  valorCategoria.textContent =
    formatarValorMoeda(totais[categoria]);
}
```

A aplicação percorre todas as categorias, inclusive aquelas que ainda possuem total igual a zero.

Isso mantém o resumo sempre organizado e previsível.

15. ENVIO DO FORMULÁRIO

```
function aoEnviarFormulario(evento: Event): void {
  evento.preventDefault();

  const tituloInformado = campoTituloDespesa.value;
  const valorInformado = campoValorDespesa.value;
  const categoriaInformada =
    campoCategoriaDespesa.value;
}
```

preventDefault() impede o comportamento padrão do formulário, que normalmente recarregaria a página.

Depois, os valores digitados são obtidos através da propriedade **value**.

```
const mensagemErro = validarFormulario(
  tituloInformado,
  valorInformado,
  categoriaInformada
);

if (mensagemErro !== null) {
  exibirMensagemErro(mensagemErro);
  return;
}
```

Primeiro o sistema valida os dados. Se houver erro, ele é exibido e a função é encerrada.

Se não houver erro, a despesa é criada, o formulário é limpo e a interface é atualizada.

16. EXCLUSÃO DE DESPESAS

```
function excluirDespesa(idDespesa: string): void {
  despesas = despesas.filter(
    (despesa) => despesa.id !== idDespesa
  );
}
```

filter() cria um novo array contendo apenas os elementos que atendem a uma condição.

Nesse caso, todas as despesas são mantidas, exceto aquela cujo id é igual ao id recebido.

17. IDENTIFICANDO O BOTÃO CLICADO

```
const botaoExcluir =
  (evento.target as HTMLElement)
    .closest<HTMLButtonElement>(
      ".despesa__botao-excluir"
    );
```

evento.target representa o elemento que recebeu o clique.

as HTMLElement é uma conversão de tipo para informar ao TypeScript como o elemento será tratado.

closest() procura o próprio elemento ou um elemento pai que corresponda ao seletor informado.

```
const itemDespesa =
  botaoExcluir.closest<HTMLLIElement>(".despesa");

const idDespesa =
  itemDespesa?.dataset.idDespesa;
```

O operador **?.** é chamado de **optional chaining**. Ele evita erro caso **itemDespesa** não exista.

Com o id encontrado, a despesa é removida e a lista e o resumo são atualizados.

18. INICIALIZAÇÃO DA APLICAÇÃO

```
function iniciarPainel(): void {
  formularioNovaDespesa.addEventListener(
    "submit",
    aoEnviarFormulario
  );

  listaDespesas.addEventListener(
    "click",
    aoClicarExcluirDespesa
  );

  atualizarResumo();
  atualizarListaDespesas();
}

iniciarPainel();
```

addEventListener() registra um evento.

Quando o formulário é enviado, a função **aoEnviarFormulario** é executada.

Quando ocorre um clique na lista, a função **aoClicarExcluirDespesa** verifica se o clique aconteceu em um botão de exclusão.

Ao final, **iniciarPainel()** é chamado para colocar toda a aplicação em funcionamento.

RESUMO DOS PRINCIPAIS CONCEITOS

Conceito	Função no projeto
const / let	Declaração de variáveis
string / number / void	Tipos básicos do TypeScript
Categoria[] / Despesa[]	Arrays tipados
Record<Categoria, number>	Objeto com totais por categoria
querySelector()	Busca elementos HTML
createElement()	Cria elementos HTML
addEventListener()	Escuta eventos do usuário
reduce()	Soma valores do array
filter()	Remove uma despesa do array
for...of	Percorre elementos
if / return	Controla decisões e fluxo
dataset	Armazena o id da despesa no HTML
closest()	Encontra elementos relacionados
spread (...)	Cria um novo array com os itens existentes
template string	Insere valores em textos com ${...}

CONCLUSÃO

Esse projeto é um bom exemplo de integração entre **HTML, CSS, JavaScript e TypeScript**. A lógica é organizada em funções pequenas e específicas: validar dados, criar despesas, calcular valores, atualizar a tela e excluir registros.

Para uma próxima evolução, os alunos poderiam adicionar: edição de despesas, armazenamento no LocalStorage, filtros por categoria, gráficos e integração com uma API ou banco de dados.